C3 - Introduction to Agents

EECE 503P/798S: Agentic Systems

Assignment: Build an Agent-Powered Business Assistant

Objective

Build a smart agent that represents a fictitious business. It should be able to:

- Answer questions about the business, drawing on a persona and a set of instructions rather than a single generic prompt,
- Route each incoming message to the right kind of handling: general question, lead collection, or feedback logging,
- Collect leads (customer name, email, notes),
- Record customer feedback or unanswered questions through tool calls,
- Run as a chatbot using the Gemini API or OpenAI (your choice),
- Deploy via Gradio.

Project Structure

```
business_bot/
│
├── me/
│   ├── about_business.pdf       # Simulated business profile (PDF)
│   └── business_summary.txt     # Short summary (TXT)
│
├── business_agent.ipynb         # Your main chatbot code (Colab or Jupyter)
├── app.py (optional)            # For deployment
├── .env                         # Contains API keys
└── requirements.txt             # Libraries used
```

Tasks and Guidelines

1. Create Your Business Identity

Invent a fictional business with:

- A name (for example, GreenLeaf Tech),
- A mission (for example, sustainable AI solutions),
- Services offered (for example, consulting, APIs),
- Team (for example, profiles of founders),
- A unique value proposition.

Write this in two formats:

- A PDF file named about_business.pdf
- A summary.txt describing your business in a few paragraphs

2. Define the Agent's Persona and Instructions

Following the core components covered in class, separate your agent's identity into two distinct parts rather than a single blob of text:

- **Persona:** who the agent is (for example, “You are Mia, the friendly virtual concierge of GreenLeaf Tech.”). This shapes tone and style.
- **Instructions:** the operational guardrails the agent must follow (for example, “Never invent pricing that is not in the business summary,” “Always ask for an email before ending a lead conversation.”).

Keep the two clearly labeled in your notebook so a reader can tell which lines are persona and which are instructions.

3. Add a Simple Router

Before your agent decides what to do, it should classify the incoming message into one of three intents: general question, lead collection, or feedback and unanswered question. This can be a lightweight LLM call, a classification prompt, or simple heuristics, your choice, as long as you can show it working. Route each intent to the appropriate handling logic or tool.

4. Create Tool Functions

You must implement at least two tool-calling functions:

- record_customer_interest(email, name, message)
- record_feedback(question), called when the chatbot does not know the answer

Both tools should log via a push or print mechanism (for example, print statements or file logging). Document each tool's schema (name, parameters, expected output) the way MCP style tool definitions are described in class.

5. System Prompt and Chat Setup

Assemble your persona and instructions into a system_prompt where the agent:

- Stays in character as the business
- Uses the summary and PDF content for answering questions
- Logs unknown questions via a tool
- Encourages leads to leave contact info

6. Agent Interaction

Use google.generativeai or openai.ChatCompletion:

- Pass user input, chat history (short-term memory), and tools to the model
- If the model calls a tool, handle it and return the response
- Otherwise, return the generated text

7. Gradio Interface

Add a Gradio ChatInterface so you can demo your bot.

8. (Optional Bonus) Reflection Step

Before your agent sends its final reply, add a lightweight reflection pass where the model briefly checks its own draft answer against the instructions (for example, did it invent information, did it forget to ask for contact details) and revises the reply if needed. Log the original draft and the revised reply so the self-correction is visible.

A second, smaller bonus is available for deploying your app to Hugging Face Spaces.

Example Use Cases

- A bakery taking orders and collecting feedback
- A tutoring service helping students pick classes
- A travel agency answering trip questions
- A SaaS product collecting bug reports

Tips

- Be creative with your business
- Ensure tools are actually called by the model when needed
- Keep tool descriptions short and helpful
- Keep the router simple; it only needs to be good enough to send messages down the right path

Submission Checklist

Submit a GitHub repo with a video showcasing the functionality, containing the following:

- business_summary.txt
- about_business.pdf
- business_agent.ipynb
- A clearly labeled persona and instructions block
- A working router that sends messages to the right handling path
- Two working tools
- API key in .env (not uploaded, of course)
- Runs successfully
- (Bonus) Reflection step with logged draft and revised replies
- (Bonus) Deployed on Hugging Face Spaces